# Emotion Detection & Learning Support Engine

## Project Description:

The Emotion Detection & Learning Support Engine is an AI-powered web application that helps students by detecting their emotions from learning-related text and providing personalized academic support. Students often experience emotions such as confusion, frustration, curiosity, or confidence while learning. This application analyses a student's written learning problem, detects the underlying emotion using two deep learning models BiLSTM and BERT, provides personalized recommendations along with AI-generated guidance through Google Gemini.

Built using Streamlit and powered by TensorFlow, PyTorch, Hugging Face Transformers, and the Google Gemini API, the platform ensures a seamless and intelligent learning support experience. The system maintains a full session history, provides an analytics dashboard with Plotly visualizations, and allows students to download a complete analysis report.

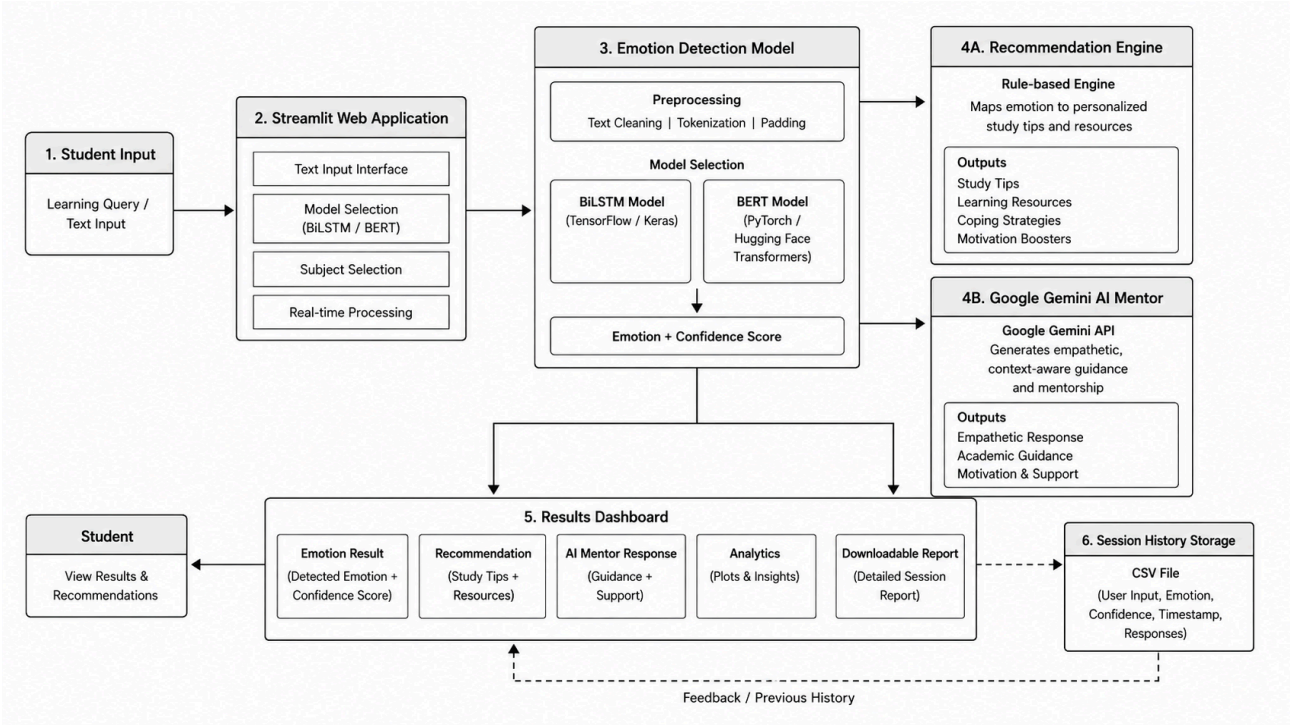
## Scenarios

**Scenario 1:** A student struggling with recursion types: "I cannot understand recursion at all, it loops infinitely in my head." They select the BiLSTM model and choose Python as their subject. The system detects the emotion as **Confused** with a high confidence score, recommends breaking the topic into smaller concepts, and the Gemini AI provides a structured, empathetic explanation of recursion tailored to their frustration.

**Scenario 2:** A student writes: "I just solved all five LeetCode problems on binary trees by myself!" They select BERT and the subject Data Structures. The system identifies the emotion as **Confident**, recommends tackling more advanced challenges, and Gemini AI responds with motivational guidance to sustain the momentum.

**Scenario 3:** A student types: "Why do we even study Operating Systems? It's so boring and useless." The BiLSTM model detects **Bored** with strong confidence, the recommendation engine suggests interactive platforms and mini-challenges, and the Gemini mentor reframes OS concepts with real-world examples to reignite interest.

# Technical Architecture:




**Description:** The Emotion Detection & Learning Support Engine uses a modular three-tier architecture. The Streamlit-based frontend presents an interactive interface for text input, model selection, and subject choice. The backend processes the text through either the BiLSTM (TensorFlow/Keras) or BERT (PyTorch / Hugging Face Transformers) model to detect emotion and compute a confidence score. The detected emotion is passed to a rule-based Recommendation Engine and simultaneously to the Google Gemini API for empathetic AI-mentor guidance. All session data is persisted in a CSV history file, and a full analytics dashboard is rendered using Plotly.

# Pre-requisites:

- • **Python 3.9+ Knowledge:** Core language for all ML, API, and web components.
- • **TensorFlow / Keras:** For building and loading the BiLSTM emotion detection model.
- • **PyTorch & Hugging Face Transformers:** For BERT-based emotion detection.
- • **Streamlit Framework:** For building the interactive web application.
- • **Google Gemini API Familiarity:** For generating AI mentor responses.
- • **NumPy, Pandas, Scikit-learn:** For data processing, label encoding, and preprocessing.
- • **Plotly:** For generating analytics visualizations.
- • **Version Control with Git:** For repository management and collaboration.
- • **Development Environment Setup:** Python virtual environment, pip, and IDE (VS Code recommended).

# Project Workflow:

## Milestone 1: Data Preparation and Preprocessing

- **Activity 1.1:** Source and understand the GoEmotions / emotions dataset with multi-label emotion columns.
- **Activity 1.2:** Map raw emotion labels (joy, anger, confusion, sadness, etc.) to five target classes: Confused, Confident, Curious, Frustrated, and Bored.
- **Activity 1.3:** Clean and preprocess text — lowercase conversion, URL/special character removal, whitespace normalization.
- **Activity 1.4:** Export the processed dataset to data/processed/processed_emotions.csv for use in model training.

## Milestone 2: BiLSTM Model Training

- **Activity 2.1:** Tokenize text using Keras Tokenizer (vocab size: 10,000) and pad sequences to max length of 50.
- **Activity 2.2:** Build the BiLSTM architecture: Embedding (128-dim) → Bidirectional LSTM (64 units) → Dropout (0.4) → Dense (64, ReLU) → Softmax output
- **Activity 2.3:** Train the model for 5 epochs with batch size 64 using Adam optimizer and categorical cross-entropy loss.
- **Activity 2.4:** Save the trained model, tokenizer, and label encoder to the saved_models/ directory.

## Milestone 3: BERT Model Training

- **Activity 3.1:** Load bert-base-uncased from Hugging Face and tokenize the processed dataset with max length 128.
- **Activity 3.2:** Fine-tune BertForSequenceClassification on a stratified 10,000-sample subset (80/20 train-test split).
- **Activity 3.3:** Train for 2 epochs with learning rate 2e-5, batch size 32, weight decay 0.01, and evaluate using accuracy.
- **Activity 3.4:** Save the fine-tuned BERT model, tokenizer, and label encoder to saved_models/bert_emotion_model/

## Milestone 4: Core Application Development

- **Activity 4.1:** Build emotion_detector.py with cached loaders (@st.cache_resource) for both BiLSTM and BERT models and implement predict functions for each.
- **Activity 4.2:** Build recommendation_engine.py with emotion-to-resource mappings for all detected emotion classes.
- **Activity 4.3:** Build gemini_service.py to call the Google Gemini API with a structured prompt containing emotion, subject, confidence, and student text.
- **Activity 4.4:** Build utils.py to generate formatted downloadable text reports and config.py for all path and environment variable configurations.

## Milestone 5: Streamlit Application Development

- **Activity 5.1:** Build the main app.py with page configuration, custom CSS, sidebar panel, model/subject selectors, and text input area.
- **Activity 5.2:** Implement the Analyze Emotion workflow: detect emotion → get recommendation → call Gemini AI → display results in metric cards.
- **Activity 5.3:** Add CSV-based session history persistence and implement the Analytics.py page with Plotly charts (pie, bar, histogram).
- **Activity 5.4:** Add the History.py page for tabular session review, the About.py info page, and integrate report download via st.download_button.

## Milestone 6: Testing and Deployment

- **Activity 6.1:** Run unit tests (test_models.py, test_preprocessing.py) to validate model prediction and text preprocessing pipelines.
- **Activity 6.2:** Set up the virtual environment, configure the .env file with the Gemini API key, and install all dependencies from requirements.txt.
- **Activity 6.3:** Launch the Streamlit application locally and test all five emotion classes across both models and multiple subjects.
- **Activity 6.4:** Validate session history saving, analytics dashboard rendering, and downloadable report generation.

# Milestone 1: Data Preparation and Preprocessing

In this milestone, the raw emotion dataset is loaded, cleaned, and transformed into a structured format suitable for training both the BiLSTM and BERT models. The goal is to map fine-grained emotion labels to five meaningful learning-state categories and produce clean, normalized text.

### Activity 1.1: Dataset Overview

The project uses the GoEmotions-style dataset containing text samples annotated with 28 emotion categories. Each row contains a text field and binary columns for each emotion. The dataset is stored at data/raw/emotions.csv.

### Activity 1.2: Emotion Mapping

The 28 raw emotion labels are mapped to 5 target classes relevant to student learning states:

- **Confused:** Mapped from the confusion label.
- **Curious:** Mapped from curiosity.
- **Confident:** Mapped from joy, optimism, approval, admiration, and pride.
- **Frustrated:** Mapped from anger, annoyance, disappointment, disapproval, and sadness.
- **Bored:** Mapped from neutral and disinterest.

### Activity 1.3: Text Cleaning

Each text sample is normalized through the following pipeline:

```
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'http\S+', '', text)
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text
```

### Activity 1.4: Dataset Export

After cleaning and emotion mapping, rows with null emotion labels are dropped. The final processed dataset containing processed_text and emotion columns is saved to data/processed/processed_emotions.csv for downstream model training.

# Milestone 2: BiLSTM Model Training

Milestone 2 focuses on training the Bidirectional LSTM model, which serves as the first emotion detection option in the application. The BiLSTM model is lightweight, fast, and achieves strong performance on the preprocessed dataset.

### Activity 2.1: Tokenization and Padding

The processed text is tokenized using Keras Tokenizer (vocabulary size: 10,000, OOV token: "<OOV>") and sequences are padded to a max length of 50 tokens.

```
VOCAB_SIZE = 10000
MAX_LENGTH = 50
tokenizer = Tokenizer(num_words=VOCAB_SIZE, oov_token='<OOV>')
tokenizer.fit_on_texts(texts)
X = pad_sequences(tokenizer.texts_to_sequences(texts),
                  maxlen=MAX_LENGTH, padding='post')
```

### Activity 2.2: Model Architecture

The BiLSTM network is constructed as a Sequential model with the following layers:

- Embedding layer (input_dim=10000, output_dim=128, input_length=50)
- Bidirectional LSTM layer (64 units)
- Dropout layer (rate=0.4)
- Dense layer (64 units, activation='relu')
- Dense output layer (softmax activation, num_classes outputs)

### Activity 2.3: Training

```
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
history = model.fit(X_train, y_train,
                    validation_data=(X_test, y_test),
                    epochs=5, batch_size=64)
```

### Activity 2.4: Saving Artifacts

The trained model, Keras tokenizer, and Scikit-learn LabelEncoder are serialized and saved:

- saved_models/bilstm_model.keras — Keras model file
- saved_models/tokenizer.pkl — Pickle serialized Keras tokenizer
- saved_models/label_encoder.pkl — Pickle serialized LabelEncoder

## Milestone 3: BERT Model Training

Milestone 3 covers the fine-tuning of a pre-trained BERT model for emotion classification. BERT (Bidirectional Encoder Representations from Transformers) provides superior contextual understanding, making it highly effective for nuanced emotion detection in student text.

### Activity 3.1: Dataset Preparation for BERT

A stratified sample of 10,000 rows is drawn from the processed dataset. The text is tokenized using BertTokenizer with padding to max_length=128 and truncation enabled. Hugging Face Dataset objects are created for efficient batched processing.

### Activity 3.2: Model Configuration

```
MODEL_NAME = 'bert-base-uncased'
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME, num_labels=NUM_LABELS)
```

### Activity 3.3: Training Arguments

```
training_args = TrainingArguments(
    output_dir='saved_models/bert_output',
    eval_strategy='epoch',
    save_strategy='epoch',
    learning_rate=2e-5,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=16,
    num_train_epochs=2,
    weight_decay=0.01,
    load_best_model_at_end=True,
    metric_for_best_model='accuracy'
)
```

### Activity 3.4: Saving BERT Artifacts

The best-checkpoint model and tokenizer are saved to saved_models/bert/ using trainer.save_model() and tokenizer.save_pretrained(). The label encoder is separately pickled to saved_models/bert_label_encoder.pkl.

## Milestone 4: Core Application Development

Milestone 4 covers building all core Python modules that power the application backend: emotion detection, recommendation, Gemini AI integration, and utility functions.

### Activity 4.1: Emotion Detector (emotion_detector.py)

Two prediction functions are implemented, both using @st.cache_resource for efficient model loading:

- **detect_emotion(text):** Loads the BiLSTM model and Keras tokenizer, tokenizes and pads the input, runs model.predict(), and returns the top predicted emotion label and confidence.
- **detect_emotion_bert(text):** Loads the BERT model and tokenizer onto the available device (CPU/CUDA), tokenizes the input, runs a torch.no_grad() forward pass, applies softmax, and returns the top predicted emotion label and confidence.

### Activity 4.2: Recommendation Engine (recommendation_engine.py)

The get_recommendation(emotion) function maps each of the nine emotion classes (Confused, Confident, Curious, Frustrated, Bored, Neutral, Happy, Sad, Anxious) to a structured, actionable learning recommendation string. Unknown emotions return a default study encouragement message.

### Activity 4.3: Gemini AI Service (gemini_service.py)

The generate_response() function builds a structured prompt embedding the student's subject, detected emotion, confidence level, and problem text, then calls the Gemini API (gemini-1.5-flash model). A template-based fallback response is returned if the API call fails.

```
prompt = f'''
You are an empathetic AI learning assistant.
Student Subject: {field}
Detected Emotion: {emotion}
Confidence: {confidence*100:.2f}%
Student Problem: {user_input}
Instructions:
1. Be supportive and encouraging.
2. Explain why the student may feel this way.
3. Give practical study advice specific to the subject.
4. Keep the response under 200 words.
5. End with one motivational sentence.
'''
```

### Activity 4.4: Utilities and Config

- **utils.py:** Provides create_report() which formats all analysis output (timestamp, subject, model, emotion, confidence, recommendation, AI guidance) into a downloadable .txt report.
- **config.py:** Centralizes all directory paths (BASE_DIR, DATA_DIR, MODEL_DIR, HISTORY_DIR, etc.) and loads the GEMINI_API_KEY from the .env file using python-dotenv.

# Milestone 5: Streamlit Application Development

Milestone 5 focuses on building the full Streamlit web application, the main interface (app.py) and the three supporting pages into a cohesive, production-ready multi-page app.

### Activity 5.1: Main App Configuration (app.py)

The main app.py establishes the application structure:

- Page config: wide layout, custom title ("Emotion Detection & Learning Support Engine"), brain emoji favicon.
- Custom CSS injected via st.markdown: dark background (#0E1117), styled metric cards with box shadows, centered titles.
- Sidebar with a feature checklist: BiLSTM, BERT, Gemini AI, Analytics, Session History, Report Download.
- Model selector (selectbox: BiLSTM / BERT) and subject selector (11 CS subjects + Other).
- Text area (height=200px) for student input.

### Activity 5.2: Emotion Analysis Workflow

On clicking Analyze Emotion:

- Input validation checks for empty text.
- Spinner shown while detecting emotion, getting recommendation, and calling Gemini AI.
- Results displayed in three metric cards (Model, Emotion, Confidence) with a progress bar.
- Learning recommendation displayed in a success box.
- Gemini AI response shown in a chat_message bubble.
- Session data appended to history/session_history.csv (Timestamp, Model, Subject, Emotion, Confidence, User Input).
- Download button for the full analysis report (.txt).\

### Activity 5.3: Analytics Dashboard (Analytics.py)

Reads session_history.csv and renders four Plotly charts:

- Pie chart — Emotion Distribution across all sessions.
- Bar chart — Subject Distribution (frequency by subject).
- Bar chart — Model Usage (BiLSTM vs BERT comparison).
- Histogram — Confidence Score Distribution (10 bins).

Overview metrics show Total Analyses, Unique Subjects, and Models Used. A full sortable session history dataframe and a CSV download button are also provided.

### Activity 5.4: History and About Pages

- **History.py:** Displays the full session history CSV as a sortable dataframe and provides a CSV download button.
- **About.py:** Static information page listing all project features, technologies, and the project objective.

# Milestone 6: Testing and Deployment

Milestone 6 focuses on testing the application end-to-end and deploying it for local and shared access.

## Activity 6.1: Unit Testing

- **test_models.py:** Verifies the Gemini API connection by calling gemini-2.0-flash with a test prompt and printing the response.
- **test_preprocessing.py:** Validates the text preprocessing pipeline by passing a sample sentence through preprocess_text() and printing the cleaned output.

## Activity 6.2: Environment Setup

### Set Up a Virtual Environment:

```
python -m venv venv
venv\Scripts\activate        # Windows
source venv/bin/activate    # Linux / macOS
pip install -r requirements.txt
```

### Configure the API Key:

Create a .env file in the project root:

```
GEMINI_API_KEY=your_gemini_api_key_here
```

## Activity 6.3: Running the Application

### Start the Streamlit Application:

```
cd "Source Code"
streamlit run app.py
```

Once running, open a browser and navigate to http://localhost:8501 to access the app.

```
  You can now view your Streamlit app in your browser.
  Local URL:  http://localhost:8501
  Network URL: http://192.168.x.x:8501
```

Test all five emotion classes (Confused, Confident, Curious, Frustrated, Bored) with both BiLSTM and BERT models across different subjects to verify correct emotion detection, AI response generation, and session history logging.

## Activity 6.4: Validation Checklist

- Session history CSV is correctly appended after each analysis.
- Analytics dashboard renders all four Plotly charts without errors.
- The downloadable report contains all fields (timestamp, model, subject, emotion, confidence, recommendation, AI guidance).
- Gemini AI fallback template is triggered correctly when the API is unavailable.
- Both BiLSTM and BERT models load from saved_models/ without errors on first run (cached thereafter).

# Exploring the Application's Web Pages:

### Home / Main Analysis Page (app.py):

Description: The main page of the Emotion Detection & Learning Support Engine serves as the primary analysis interface. It features a dark background (#0E1117) theme with custom CSS-styled metric cards. The page includes a sidebar with a feature checklist, a model selector dropdown (BiLSTM / BERT), an 11-subject selector, a student text input area, and the full analysis results panel — all on one seamless page.

### Input Section:

A text area (height: 200px) accepts the student's learning problem description (e.g., "I cannot understand recursion..."). Above it, a selectbox lets the student choose between BiLSTM and BERT models, and another selectbox allows subject selection from Python, Java, Data Structures, Algorithms, Operating Systems, DBMS, Computer Networks, Machine Learning, Artificial Intelligence, Cloud Computing, and Other.

### Results Section:

After clicking Analyze Emotion, a spinner indicates processing. Results are displayed in three dark-styled metric cards: Model Used, Detected Emotion, and Confidence Score. A progress bar visualizes the confidence percentage. The learning recommendation appears in a green success box, and the AI Mentor response from Gemini is shown in a chat_message bubble. A download button exports the full analysis report as a .txt file.

### Analytics Dashboard (Analytics.py):

Description: The Analytics page reads the session_history.csv file and renders a comprehensive visual dashboard. Overview metric tiles show Total Analyses performed, Unique Subjects covered, and Models Used. Four Plotly charts are displayed: a pie chart for Emotion Distribution, a bar chart for Subject Distribution, a color-coded bar chart comparing BiLSTM vs BERT Usage, and a histogram of Confidence Score distribution. A full sortable session dataframe and a CSV export button are provided at the bottom.

### Session History (History.py):

Description: A simple, focused page that loads the session_history.csv file and displays it as a full-width, sortable Streamlit dataframe sorted by timestamp (most recent first). Every row shows the Timestamp, Model used, Subject, detected Emotion, Confidence percentage, and the original User Input text. A download button allows export of the complete history as a CSV file.

**About Page (About.py):**

Description: A static information page providing a project overview, listing all key features (BiLSTM/BERT emotion detection, AI mentor, analytics, history, reports), the full technology stack (Streamlit, TensorFlow, PyTorch, Hugging Face Transformers, Google Gemini API, Plotly, Pandas), and the project objective of providing personalized, emotion-aware academic guidance to improve the student learning experience.

# Conclusion:

The Emotion Detection & Learning Support Engine is an AI-driven learning platform designed to connect students with personalized learning advice through the lens of emotion. The system, which leverages two deep learning architectures (a lightweight BiLSTM model for fast inference, and a fine-tuned BERT model for high-accuracy contextual understanding), provides reliable emotion classification for learners across five emotion states: Confused, Confident, Curious, Frustrated, and Bored.

By incorporating Google Gemini AI as an empathetic mentor layer, the platform transforms from just being a mere classifier, enabling it to deliver subject-specific, emotionally attuned advice that aligns with the student's current stage. Accessible and intuitive with the Streamlit web interface and downloadable reports and an analytics dashboard for continuous tracking of learning patterns over time.

The project highlights how targeted AI, when applied in a structured framework, can impact and enhance the learning experience for students through the use of NLP, deep learning, and generative AI. In the future, the platform holds great potential for growth, including the ability to detect emotions in voice data, providing multi-language support for regional learners, enabling speech emotion recognition, creating personalized learning paths, rolling out to a cloud-based platform, integrating with databases for user accounts, enabling mobile app support, and real-time integration with classrooms for teachers.

As it continues to evolve with the addition of more complex and varied training data, and further development of features on the "front-end" of the system that make it easier to use and more personalized, the Emotion Detection & Learning Support Engine is poised to become a full-blown AI-powered academic assistant for students of any age